

fied. TensorFlow too has handy annotation tools called labelling which generates an XML file for you with some simple codes.

Next, all the images can only be taken by the system if they are present in the TFRecord file format. This is an important part of the data preparation methodology and will require you to convert the XML file we obtained from labelling to CSV. This can be done using `xml_to_csv.py` and then converting the `generate_tfrecord.py` to produce the corresponding TFRecord file.

When you feel like you've got all the data necessary to train the model, before starting the project, make sure that the paths for the images folder and the output csv folder are both changed to avoid the system getting confused.

```
• image_path = os.path.join(os.getcwd(), 'your image path'.
format(directory))
• xml_df.to_csv('your csv folder '.format(directory),
index=None)
• def main():
•     image_path = os.path.join(os.getcwd(), 'annotations')
•     xml_df = xml_to_csv(image_path)
•     xml_df.to_csv('raccoon_labels.csv', index=None)
•     print('Successfully converted xml to csv.')
```

After inputting the code above, the images would be converted to a CSV format which should then be changed to the TFRecord format by the following code in python:-

```
• $ python generate_tfrecord.py -
• csv_input=object_data/train_labels.csv --output_
path=object.data/train.record
• # TO-DO replace this with label map
• def class_text_to_int(row_label):
•     print(row_label)
•     if row_label == 'Your object name':
•         return 1
•     else:
•         None
```

Take note that the system may return some errors if the Protobuf is not properly read by the system. In case if you get any, simply troubleshoot by inputting download commands to the system for buffers that will remove any errors.